

Day 25–26: Exception Handling

- try-catch
- finally
- throw
- throws
- Custom exceptions

1. First: What is an Exception?

Before learning try-catch, throw, etc., manam first **Exception ante enti?** understand cheyyali.

Simple meaning

Exception = Program run avuthunnappudu vachche unexpected problem/error.

Example:

```
int a = 10;
```

```
int b = 0;
```

```
int result = a / b;
```

```
System.out.println(result);
```

Manam 10 / 0 chestunnam.

Mathematically zero tho division possible kaadu.

Program run chesthe:

Exception in thread "main" java.lang.ArithmeticException: / by zero

So, program normal ga continue avvadu.

2. Error vs Exception

Beginner ga ikkada confusion ravachu.

Error

Usually serious problem which application itself cannot reasonably handle.

Examples:

OutOfMemoryError

StackOverflowError

Exception

Program execution to occur any problem, which we can often **handle**.

Examples:

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException

NumberFormatException

IOException

Simple ga:

Exception = "Something went wrong while program is running, but we may be able to handle it."

3. Why Exception Handling?

Suppose ATM program undi.

User ₹10,000 withdraw cheyyali.

But account balance ₹2,000 only.

Instead of program suddenly crash avvadam, manam user ki:

Insufficient balance

ani meaningful message ivvachu.

That is the purpose of **Exception Handling**.

Without exception handling

Program starts

↓

Some problem occurs



Program terminates

With exception handling

Program starts



Some problem occurs



Exception detected



Handle the exception



Program continues

4. Important Exception Handling Keywords

Java lo main exception-handling keywords:

Keyword Meaning

try Risky code ni place cheyyadaniki

catch Exception ni handle cheyyadaniki

finally Usually always execute avvalsina code

throw Manually exception create/raise cheyyadaniki

throws Method exception throw cheyyachu ani declare cheyyadaniki

Memory trick:

try → Try this code

catch → Problem vaste catch it

finally → Finally, execute this

throw → Nene exception throw chestha

throws → Ee method exception throw cheyyachu

5. try-catch

Idi exception handling lo **most important concept**.

Basic structure:

```
try {  
    // risky code  
}  
catch (ExceptionType e) {  
    // handling code  
}
```

Example:

```
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            int result = 10 / 0;  
            System.out.println(result);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
  
    }  
}
```

Output:

Cannot divide by zero

6. try ante enti?

try block lo manam **exception occur avvachu ani suspect chestunna code** pedatham.

```
try {  
    int result = 10 / 0;  
}
```

Meaning:

"Java, ee code execute cheyyi. Problem vaste nenu handle chestha."

7. catch ante enti?

catch block exception ni **handle** chestundi.

```
catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

Meaning:

"ArithmeticException vachindaa? Sare, danini nenu handle chestha."

8. e ante enti?

Ikkada:

```
catch (ArithmeticException e)
```

e is a **reference variable**.

It refers to the exception object.

Example:

```
catch (ArithmeticException e) {  
    System.out.println(e);  
}
```

Output something like:

java.lang.ArithmeticException: / by zero

Manam:

```
e.getMessage()
```

kuda use cheyyachu.

```
catch (ArithmeticException e) {
```

```
    System.out.println(e.getMessage());  
}
```

Output:

/ by zero

9. Exception Object ante enti?

Java lo exception kuda **object**.

For example:

ArithmeticException

is a class.

When exception occurs, Java creates an object related to that exception.

Conceptually:

ArithmeticException object

↓

e

So:

catch (ArithmeticException e)

means:

"ArithmeticException type object ni e ane reference tho receive cheyyi."

10. What happens internally?

Let's understand this carefully.

Code:

```
try {  
    int a = 10 / 0;  
    System.out.println("Hello");  
}  
catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

```
}
```

```
System.out.println("Program continues");
```

Execution:

Step 1

Java enters try.

```
int a = 10 / 0;
```

Step 2

Exception occurs.

ArithmeticException

Step 3

Java immediately stops the remaining statements in that try block.

So:

```
System.out.println("Hello");
```

execute avvadu.

Step 4

Java searches for matching catch.

```
catch (ArithmeticException e)
```

Match ayyindi.

Step 5

Catch executes:

Cannot divide by zero

Step 6

After catch, program continues.

Program continues

Final output:

Cannot divide by zero

Program continues

11. Very Important: Exception occurs after some code

Example:

```
try {  
    System.out.println("A");  
  
    int x = 10 / 0;  
  
    System.out.println("B");  
}  
catch (ArithmeticException e) {  
    System.out.println("C");  
}
```

Output:

A

C

Why not B?

Because exception occurred before B.

Once exception occurs:

Remaining try code

↓

SKIPPED

12. Multiple catch blocks

Manam different types of exceptions handle cheyyachu.

```
try {  
    int[] arr = {10, 20, 30};  
  
    System.out.println(arr[5]);  
}
```

```
catch (ArithmeticException e) {  
    System.out.println("Arithmetic problem");  
}  
catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Array index problem");  
}
```

Here array index problem kabatti second catch execute avtundi.

Output:

Array index problem

13. Why multiple catch?

Different problems ki different responses kavali.

Example:

ArithmeticException

↓

"Cannot divide by zero"

ArrayIndexOutOfBoundsException

↓

"Invalid array index"

NumberFormatException

↓

"Please enter a valid number"

So multiple catch blocks useful.

14. Order of catch blocks — VERY IMPORTANT

Suppose:

```
try {
```

```

    // code
}
catch (Exception e) {
    // general
}
catch (ArithmeticException e) {
    // specific
}

```

This is **wrong**.

Why?

Exception is parent/general type.

ArithmeticException is more specific.

If Exception first vasthe, every exception ni adi catch cheyyagaladu.

So second catch unreachable.

Correct:

```

try {
    // code
}
catch (ArithmeticException e) {
    // specific
}
catch (Exception e) {
    // general
}

```

Rule

Specific catch first, general catch later.

15. Exception itself

Manam ila kuda rayachu:

```
catch (Exception e) {  
    System.out.println("Something went wrong");  
}
```

Exception is a general parent class for many exceptions.

Example:

```
try {  
    int x = 10 / 0;  
}  
catch (Exception e) {  
    System.out.println("Something went wrong");  
}
```

This can catch `ArithmeticException`.

But beginners ki specific exception use cheyyadam better when possible:

```
catch (ArithmeticException e)  
because it clearly tells what problem we're handling.
```

16. finally

Now second important keyword.

Definition

finally block lo normally **must-run cleanup code** pedatham.

Syntax:

```
try {  
    // risky code  
}  
catch (Exception e) {  
    // handle  
}  
finally {  
    // cleanup code
```

```
}
```

Example:

```
try {  
    int result = 10 / 2;  
    System.out.println(result);  
}  
catch (ArithmeticException e) {  
    System.out.println("Error");  
}  
finally {  
    System.out.println("Finally executed");  
}
```

Output:

5

Finally executed

17. Exception vachina kuda finally

```
try {  
    int result = 10 / 0;  
}  
catch (ArithmeticException e) {  
    System.out.println("Error handled");  
}  
finally {  
    System.out.println("Finally executed");  
}
```

Output:

Error handled

Finally executed

So exception vachina:

catch

↓

finally

18. Exception raakapoina finally

```
try {  
    int result = 10 / 2;  
    System.out.println(result);  
}  
catch (ArithmeticException e) {  
    System.out.println("Error");  
}  
finally {  
    System.out.println("Finally");  
}
```

Output:

5

Finally

So:

Exception vachina → finally

Exception raakapoina → finally

19. Why finally is useful?

Suppose manam file open chesam.

Open file

↓

Read file

↓

Problem?

Problem vachina or rakapoyina file ni close cheyyali.

Similarly:

- file close cheyyadam
- database connection cleanup
- resources release cheyyadam

lanti cleanup operations kosam finally historically very important.

Modern Java lo resources kosam **try-with-resources** kuda use chestaru; adi later topic.

20. Can we have try without catch?

Yes, but finally undali.

```
try {  
    System.out.println("Hello");  
}  
finally {  
    System.out.println("Finally");  
}
```

Valid.

But:

```
try {  
    System.out.println("Hello");  
}
```

alone is **invalid**.

try ki either:

catch

or

finally

undali.

21. throw

Now very important difference.

throw means:

Manually exception ni throw cheyyadam.

Example:

```
throw new ArithmeticException("Invalid operation");
```

Here maname exception create chesi throw chestunnam.

22. Why manually throw?

Suppose bank application.

balance = 1000

withdraw = 5000

Java automatically "Insufficient balance" exception create cheyyadu.

So maname condition check chesi exception throw cheyyachu.

```
if (amount > balance) {  
    throw new Exception("Insufficient balance");  
}
```

But checked exception kabatti method declaration etc. required. Beginner-friendly custom exception example ni later chuddam.

23. Simple throw example

```
public class Main {  
    public static void main(String[] args) {  
  
        int age = 15;  
  
        if (age < 18) {  
            throw new ArithmeticException("Age must be 18 or above");  
        }  
    }  
}
```

```

    }

    System.out.println("Eligible");
}
}

```

Output:

Exception in thread "main" java.lang.ArithmeticException: Age must be 18 or above
Because maname exception throw chesam.

24. throw new meaning

Look at:

```
throw new ArithmeticException("Age must be 18 or above");
```

Break it:

new

Creates an object.

```
new ArithmeticException(...)
```

means:

Create an ArithmeticException object.

Message

"Age must be 18 or above"

exception ki message.

throw

That exception object ni Java ki throw cheyyamani cheptundi.

So:

new

↓

Create exception object

throw

↓

Send/raise that exception

25. throw with try-catch

Usually practical usage:

```
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            int age = 15;  
  
            if (age < 18) {  
                throw new Exception("Age must be 18 or above");  
            }  
  
            System.out.println("Eligible");  
        }  
        catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output:

Age must be 18 or above

Flow:

age < 18

↓

throw

↓

Exception object



catch



message printed

26. throws

Now throw and throws ni confuse avvakunda very clearly understand cheddham.

throw

Actually exception ni throw chestundi.

Example:

```
throw new Exception("Problem");
```

throws

Method declaration lo exception possibility ni announce chestundi.

Example:

```
void test() throws Exception {  
    // code  
}
```

Meaning:

"Ee method execute chestunnappudu Exception ravachu."

27. throws example

```
public class Main {  
  
    static void test() throws Exception {  
        throw new Exception("Something went wrong");  
    }  
  
    public static void main(String[] args) {
```

```
try {  
    test();  
}  
catch (Exception e) {  
    System.out.println(e.getMessage());  
}  
}
```

Output:

Something went wrong

Let's understand flow.

main()

↓

test()

↓

throw exception

↓

test() says throws Exception

↓

exception goes back to caller

↓

main() catches it

28. Why throws?

Suppose method:

```
static void readFile() throws IOException {  
    // file operation  
}
```

Method itself may not handle the exception.

It tells the caller:

"Naa method lo IOException ravachu. Nuvvu handle cheyyali."

So responsibility is passed to the caller.

29. throw vs throws

This is an **interview favorite**.

throw

Used to actually throw exception

Used inside method/block

Followed by exception object

Usually one exception at a time

Example:

throw

```
throw new Exception("Error");
```

throws

```
void test() throws Exception {
```

```
}
```

Memory trick:

throw = action

throws = warning/declaration

30. Can throws have multiple exceptions?

Yes.

```
void test() throws IOException, SQLException {
```

```
    // code
```

```
}
```

Meaning:

This method may throw either IOException or SQLException.

31. Checked vs Unchecked Exceptions

Now we need this concept because throws and custom exceptions are important.

Java exceptions broadly:

Throwable

|

+--- Error

|

+--- Exception

|

+--- RuntimeException

Checked Exception

Compiler checks these.

Examples:

IOException

SQLException

ClassNotFoundException

Compiler generally forces you to either:

handle using try-catch

or:

declare using throws

Example:

```
void test() throws Exception {  
}
```

32. Unchecked Exception

These are generally subclasses of RuntimeException.

Examples:

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException

NumberFormatException

Compiler does not force you to handle them.

Example:

```
int x = 10 / 0;
```

You don't have to write:

throws ArithmeticException

But you **can** explicitly declare it if you want.

33. Checked vs Unchecked — easy memory

Checked

Compiler says:

"Handle it or declare it!"

Unchecked

Compiler says:

"Your choice."

But good programming practice is still to prevent/handle problems appropriately.

34. Custom Exceptions

Now the most interesting part.

What is a custom exception?

Java already gives us:

ArithmeticException

NullPointerException

IOException

But our application may need its own meaningful exceptions.

Example:

InsufficientBalanceException

InvalidAgeException

InvalidPasswordException

InvalidMarksException

InsufficientStockException

These are **custom exceptions**.

35. Why custom exceptions?

Suppose bank application.

We could write:

```
throw new Exception("Insufficient balance");
```

It works, but it's generic.

Better:

```
throw new InsufficientBalanceException("Insufficient balance");
```

Now the exception itself communicates the business problem.

36. Creating a Custom Exception

Basic structure:

```
class InvalidAgeException extends Exception {
```

```
    InvalidAgeException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

Let's understand **every single word**.

37. class InvalidAgeException

We're creating our own class:

```
class InvalidAgeException
```

Name itself tells what went wrong.

38. extends Exception

```
class InvalidAgeException extends Exception
```

Means:

Our InvalidAgeException is an Exception.

Because it extends Exception, it becomes a custom checked exception.

39. Constructor

```
InvalidAgeException(String message) {  
    super(message);  
}
```

We accept a message.

Example:

```
new InvalidAgeException("Age must be 18 or above");
```

That message goes into:

String message

Then:

```
super(message);
```

passes the message to the parent Exception class.

40. Complete Custom Exception Example

```
class InvalidAgeException extends Exception {
```

```
    InvalidAgeException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {
```

```

static void checkAge(int age) throws InvalidAgeException {

    if (age < 18) {
        throw new InvalidAgeException("Age must be 18 or above");
    }

    System.out.println("Eligible to vote");
}

public static void main(String[] args) {

    try {
        checkAge(15);
    }
    catch (InvalidAgeException e) {
        System.out.println(e.getMessage());
    }
}
}

```

Output:

Age must be 18 or above

41. Understand this program like a story

We have:

class InvalidAgeException extends Exception

Meaning:

"Nenu oka new exception create chestunna."

Then:

```
checkAge(int age)
```

Age check chestundi.

If:

```
age < 18
```

then:

```
throw new InvalidAgeException(...)
```

So:

Age invalid

↓

Create InvalidAgeException object

↓

throw it

↓

checkAge() declares throws InvalidAgeException

↓

main() calls checkAge()

↓

try-catch handles it

42. Custom Exception with Bank Example

Let's create a more realistic example.

```
class InsufficientBalanceException extends Exception {
```

```
    InsufficientBalanceException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
public class Main {
```

```

static void withdraw(int balance, int amount)
    throws InsufficientBalanceException {

    if (amount > balance) {
        throw new InsufficientBalanceException(
            "Insufficient balance"
        );
    }

    System.out.println("Withdrawal successful");
}

public static void main(String[] args) {

    try {
        withdraw(1000, 5000);
    }
    catch (InsufficientBalanceException e) {
        System.out.println(e.getMessage());
    }
}

```

Output:

Insufficient balance

43. Real-world flow

Imagine ATM.

User:

Withdraw ₹5000

Balance:

₹1000

System:

if (amount > balance)

Condition true.

So:

throw new InsufficientBalanceException(...)

Then catch:

catch (InsufficientBalanceException e)

and display:

Insufficient balance

This is much better than application crashing.

44. Custom Unchecked Exception

Custom exception can also extend RuntimeException.

Example:

```
class InvalidAgeException extends RuntimeException {
```

```
    InvalidAgeException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

Now it is an **unchecked custom exception**.

We don't have to write:

throws InvalidAgeException

though we can.

Example:

```
class InvalidAgeException extends RuntimeException {
```

```

InvalidAgeException(String message) {
    super(message);
}
}

public class Main {

    static void checkAge(int age) {

        if (age < 18) {
            throw new InvalidAgeException("Invalid age");
        }

        System.out.println("Valid age");
    }

    public static void main(String[] args) {

        checkAge(15);
    }
}

```

45. Checked custom vs Unchecked custom

Checked custom exception

class MyException extends Exception

Compiler expects handling/declaration.

Usually:

throws MyException

or try-catch.

Unchecked custom exception

class MyException extends RuntimeException

Compiler doesn't force handling.

46. finally with return

Interesting interview concept.

```
public static int test() {  
  
    try {  
        return 10;  
    }  
    finally {  
        System.out.println("Finally");  
    }  
}
```

Output:

Finally

Returned value:

10

So finally executes before method actually completes.

47. Avoid return inside finally

Technically possible:

```
try {  
    return 10;  
}  
finally {  
    return 20;  
}
```

```
}
```

This causes confusing behavior because the finally return can override the earlier return.

So practical rule:

Don't use return inside finally unless you have a very specific reason.

48. Can we have try-catch-finally together?

Yes.

```
try {  
    // risky code  
}  
catch (Exception e) {  
    // handling  
}  
finally {  
    // cleanup  
}
```

This is the most common complete structure.

49. Nested try-catch

try inside another try is possible.

```
try {  
  
    try {  
        int x = 10 / 0;  
    }  
    catch (ArithmeticException e) {  
        System.out.println("Inner catch");  
    }  
}
```

```
}  
catch (Exception e) {  
    System.out.println("Outer catch");  
}
```

Inner catch handles the exception, so outer catch doesn't need to handle it.

50. Exception propagation

Very important concept.

Suppose:

```
static void method3() {  
    int x = 10 / 0;  
}
```

```
static void method2() {  
    method3();  
}
```

```
static void method1() {  
    method2();  
}
```

Exception occurs in:

method3()

If method3 doesn't handle it, Java looks to caller:

method3()

↓

method2()

↓

method1()

↓

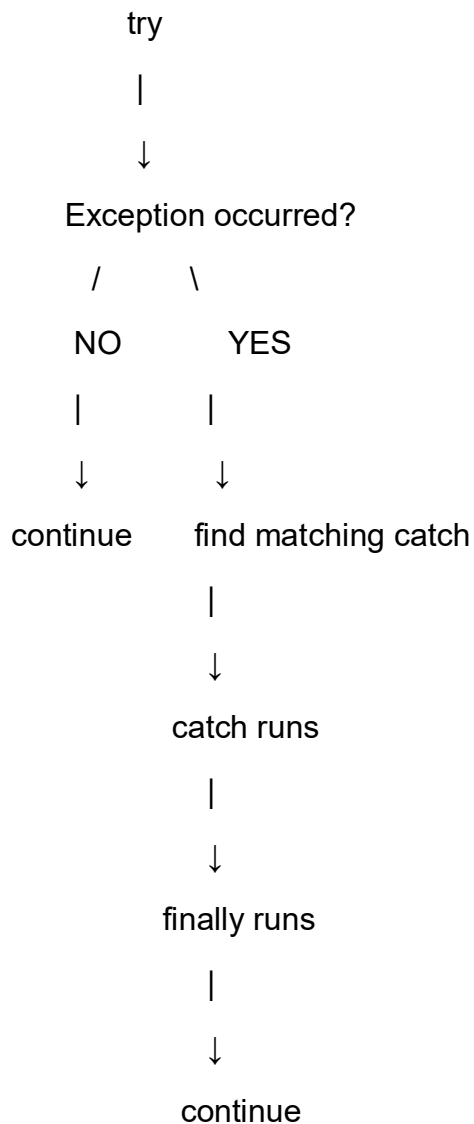
main()

This is called **exception propagation**.

Simple meaning:

Exception handle cheyyakapothe, adi caller methods ki move avtundi until matching handler dorike varaku.

51. Exception handling flow — memorize this



52. Common exceptions you should know

1. ArithmeticException

```
int x = 10 / 0;
```

2. NullPointerException

```
String name = null;
```

```
System.out.println(name.length());
```

3. ArrayIndexOutOfBoundsException

```
int[] arr = {10, 20};
```

```
System.out.println(arr[5]);
```

4. NumberFormatException

```
String value = "abc";
```

```
int x = Integer.parseInt(value);
```

5. StringIndexOutOfBoundsException

```
String name = "Java";
```

```
System.out.println(name.charAt(10));
```

53. getMessage()

Useful method:

```
e.getMessage()
```

Example:

```
catch (Exception e) {  
    System.out.println(e.getMessage());  
}
```

It generally gives the exception's descriptive message.

54. printStackTrace()

Another useful method:

```
e.printStackTrace();
```

This prints detailed information about where the exception happened, including the stack trace.

Example:

```
try {  
    int x = 10 / 0;  
}  
catch (ArithmeticException e) {  
    e.printStackTrace();  
}
```

Developers use stack traces for debugging.

55. toString() vs getMessage()

Suppose:

```
throw new Exception("Something went wrong");
```

Then:

```
e.getMessage()
```

generally gives:

Something went wrong

Whereas:

```
e.toString()
```

generally includes the exception type and message:

```
java.lang.Exception: Something went wrong
```

56. One complete example

Let's combine everything.

```
class InvalidAgeException extends Exception {
```

```
    InvalidAgeException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {
```

```
    static void checkAge(int age)  
        throws InvalidAgeException {
```

```
        if (age < 18) {  
            throw new InvalidAgeException(  
                "Age must be 18 or above"  
            );  
        }
```

```
        System.out.println("Eligible");  
    }
```

```
    public static void main(String[] args) {
```

```
        try {  
            checkAge(15);  
        }  
        catch (InvalidAgeException e) {  
            System.out.println("Exception: " + e.getMessage());  
        }
```

```
    }  
    finally {  
        System.out.println("Age verification completed");  
    }  
}  
}
```

Output:

Exception: Age must be 18 or above

Age verification completed

57. Understand the complete flow

This one example contains:

try

```
try {  
    checkAge(15);  
}
```

Risky operation.

throw

Inside method:

```
throw new InvalidAgeException(...);
```

Manually creates and throws exception.

throws

```
static void checkAge(int age)  
    throws InvalidAgeException
```

Method declares that it may throw this exception.

catch

```
catch (InvalidAgeException e)
```

Handles exception.

finally

```
finally {  
    System.out.println(...);  
}
```

Cleanup/final action.

58. Most important difference: throw vs throws

Remember this exact mental picture:

```
METHOD  
|  
throws Exception  
|  
↓  
"I may throw it"
```

Inside method:

```
if (problem) {  
    throw new Exception();  
}
```

Meaning:

"I am actually throwing it NOW."

So:

throws = declaration

throw = actual throwing

59. Real-life analogy

Imagine a school.

try

Student says:

"Sir, nenu ee task try chestha."

catch

Teacher says:

"Problem vachinda? Okay, nenu handle chestha."

finally

Teacher says:

"Whatever happens, classroom cleanup complete cheyyali."

throw

Student himself says:

"This situation is invalid, so I'm raising a problem."

throws

Student tells teacher beforehand:

"Sir, ee task lo problem ravachu."

That's the easiest way to remember them.

60. Common mistakes beginners make

Mistake 1

Writing:

```
try {  
}
```

alone.

✗ Invalid.

Need:

catch

or

finally

Mistake 2

Putting general catch before specific catch.

✗

catch (Exception e)

catch (ArithmeticException e)



catch (ArithmeticException e)

catch (Exception e)

Mistake 3

Confusing:

throw

and

throws

Remember:

throw → actual exception

throws → declaration

Mistake 4

Thinking finally handles exception.

It doesn't.

catch handles exception.

finally is mainly for code that should run afterward, especially cleanup.

Mistake 5

Thinking throws handles exception.

It doesn't.

throws **declares** the possibility; it doesn't itself handle the exception.

61. Interview Questions You Must Know

Q1. What is exception?

An exception is an abnormal condition that occurs during program execution and can disrupt normal program flow.

Q2. Why exception handling?

To handle runtime problems gracefully and prevent abnormal termination where appropriate.

Q3. What is try?

It contains code that may cause an exception.

Q4. What is catch?

It handles a matching exception thrown from the associated try block.

Q5. What is finally?

A block generally used for cleanup code that should execute whether an exception occurs or not.

Q6. What is throw?

Used to explicitly throw an exception.

`throw new Exception();`

Q7. What is throws?

Used in a method declaration to declare exceptions that the method may pass to its caller.

`void test() throws IOException`

Q8. Can we have multiple catch blocks?

Yes.

`catch (ArithmeticException e)`

`catch (NullPointerException e)`

`catch (Exception e)`

Q9. Can try exist without catch?

Yes, if it has finally.

```
try {  
}  
finally {  
}
```

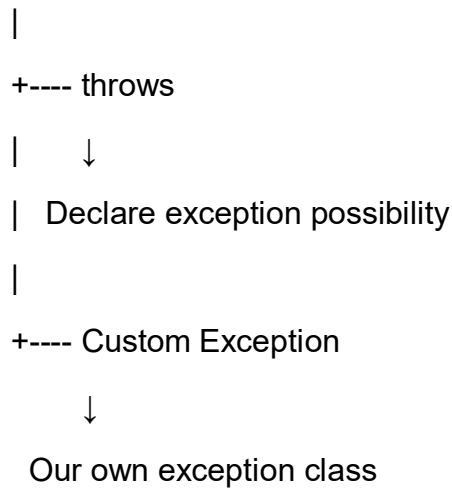
Q10. What is custom exception?

An application-specific exception created by defining a class that extends Exception or RuntimeException.

62. Final Cheat Sheet □

EXCEPTION HANDLING

```
|  
+---- try  
|   ↓  
|   Risky code  
|  
+---- catch  
|   ↓  
|   Handle exception  
|  
+---- finally  
|   ↓  
|   Cleanup / final action  
|  
+---- throw  
|   ↓  
|   Manually throw exception
```



One-line meanings

Keyword	Easy meaning
try	Try this risky code
catch	Catch and handle the problem
finally	Run this final/cleanup code
throw	I am throwing an exception
throws	This method may throw an exception
Custom Exception	Our own application-specific exception

Day 25–26 Practice Order

Don't just read. Practice in this order:

Level 1 — Basic

1. Divide two numbers using try-catch
2. Handle array index exception
3. Handle null pointer exception
4. Handle invalid number conversion

Level 2 — finally

5. Write try-catch-finally
6. Check finally when exception occurs
7. Check finally when exception doesn't occur

Level 3 — throw

8. Throw exception if age < 18
9. Throw exception if marks are outside 0–100
10. Throw exception if withdrawal amount exceeds balance

Level 4 — throws

11. Create a method with throws
12. Call that method from main() and handle it

Level 5 — Custom Exceptions

13. InvalidAgeException
14. InsufficientBalanceException
15. InvalidMarksException
16. InsufficientStockException